

实验二 基本图形生成算法

一、实验目的

- 1.掌握直线生成算法。
- 2.掌握圆和椭圆生成算法。

二、实验内容

- 1、 利用 OpenGL 绘图函数生成直线;
- 2、 利用 OpenGL 生成圆和椭圆;

三、实验环境

PC 微机

Windows 10 操作系统

CLion 集成开发环境

四、实验步骤

1. 根据 bresenham 算法, 绘制直线;
2. 根据中点画线算法, 绘制直线;
3. 根据 bresenham 画圆算法, 绘制圆;
4. 根据中点画圆算法, 绘制圆

五、实验结果

首先, 我自定义了 draw_pixel 函数

```
void draw_pixel(int ix, int iy, int value) {  
    glBegin( mode: GL_POINTS);  
    glVertex2i( x: ix, y: iy);  
    glEnd();  
}
```

1. 中点画线法

预处理步骤

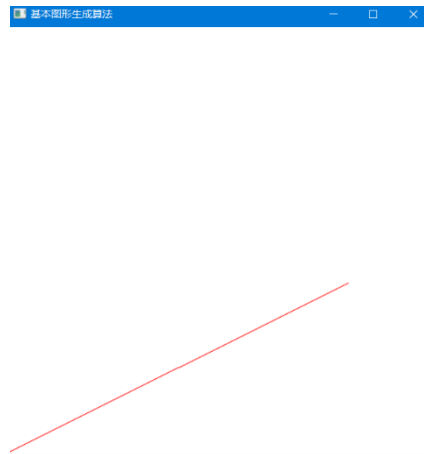
$$\begin{aligned}\Delta x &= x_n - x_1 \\ \Delta y &= y_n - y_1 \\ len &= (\max(\Delta x, \Delta y))\end{aligned}$$

算法

```
while not in end point:
    if  $D_{initial} < 0$ 
         $(x_k, y_k) \rightarrow (x_k + 1, y_k)$ 
         $D := D + 2 * \Delta y$ 
    else
         $(x_k, y_k) \rightarrow (x_k + 1, y_k + 1)$ 
         $D := D + \Delta D$ 
```

```
void mid_point_draw_line(int x1, int y1, int x2, int y2) {
    int dy = y2 - y1;          delta x, delta y
    int dx = x2 - x1;
    int D = 2 * dy - dx; D_initial
    int delta_d = 2 * dy - 2 * dx; delta D
    int xk = x1, yk = y1;
    while (xk != x2 && yk != y2) { 迭代循环
        xk++;
        if (D < 0) {
            D += 2 * dy;
        } else {
            yk++;
            D += delta_d;
        }
        draw_pixel( ix: xk, iy: yk, value: BLACK);
        std::cout << "(" << xk << ", " << yk << ")" << std::endl;
    }
}
```

结果：



2. Bresenham 算法

前提条件

- 直线斜率 m 满足 $0 < m < 1$
- 在其他区域需要额外处理

输入

$(x_1, y_1), (x_n, y_n)$

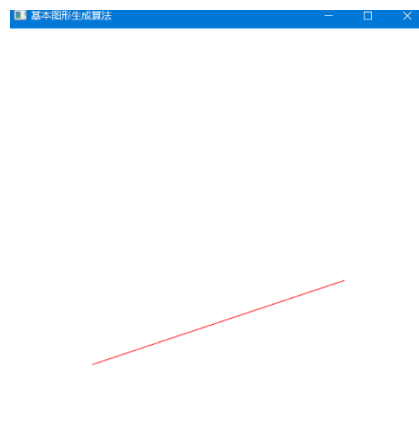
预处理步骤

$$\begin{aligned}\Delta x &= x_n - x_1 \\ \Delta y &= y_n - y_1 \\ P_{initial} &= 2\Delta y - \Delta x\end{aligned}$$

算法

```
while not in end point:
    if  $P_{initial} < 0$ 
         $(x_k, y_k) \rightarrow (x_k + 1, y_k)$ 
         $D := D + 2 * \Delta y$ 
    else
         $(x_k, y_k) \rightarrow (x_k + 1, y_k + 1)$ 
         $D := D + 2 * (\Delta y - \Delta x)$ 
```

```
void bres(int x1, int y1, int x2, int y2) {
    int dy = y2 - y1;
    int dx = x2 - x1;
    int p = 2 * dy - dx;    p_initial
    int d1 = 2 * dy, d2 = 2 * (dy - dx);
    int xk = x1, yk = y1;
    int times = 0;
    while ((xk != x2 && yk != y2) || (times < dx)) {
        times++;
        xk++;
        if (p < 0) {
            p += d1;
        } else {
            p += d2;
            yk++;
        }
        draw_pixel(xk, yk, value: BLACK); 自定义绘图函数
        std::cout << "(" << xk << ", " << yk << ")" << std::endl;
    }
}
```



3. 中点画圆算法:

输入

$(x_0, y_0) R$

预处理步骤

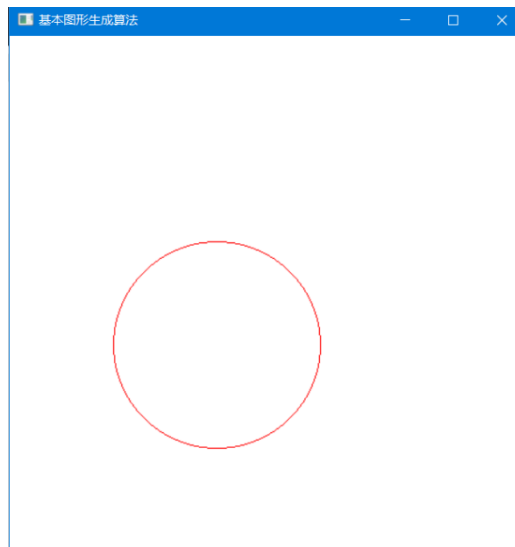
$$\begin{aligned}x_k &:= 0 \quad y_k = R \\ P_{\text{initial}} &= 1 - R\end{aligned}$$

算法

```
while  $x_c < y_c$ 
  if  $P_{\text{initial}} < 0$ 
     $(x_k, y_k) \rightarrow (x_k + 1, y_k)$ 
     $P := P + 2 * x_k + 1$ 
  else
     $(x_k, y_k) \rightarrow (x_k + 1, y_k - 1)$ 
     $P := P - 2 * y_k + 2 * x_k + 1$ 
```

```
void mid_point_circle_drawing(int x, int y, int r) {
    int xc = 0, yc = r;          初始坐标
    int p = 1 - r;
    draw_whole_circle(x, y, xc, yc); drawCircle的作用是绘制 给定一点 (x,y),
    while (xc < yc) {             绘制该点在八个卦限的对应点
        xc++;
        if (p < 0) {
            p = p + 2 * xc + 1;
        } else {
            yc--;
            p = p - 2 * yc + 2 * xc + 1;
        }
        draw_whole_circle(x, y, xc, yc);
    }
}
```

```
void draw_whole_circle(int x, int y, int xc, int yc) {
    draw_pixel( ix: xc + x, iy: yc + y, value: 0);
    draw_pixel( ix: yc + x, iy: xc + y, value: 0);
    draw_pixel( ix: yc + x, iy: -xc + y, value: 0);
    draw_pixel( ix: xc + x, iy: -yc + y, value: 0);
    draw_pixel( ix: -xc + x, iy: -yc + y, value: 0);
    draw_pixel( ix: -yc + x, iy: -xc + y, value: 0);
    draw_pixel( ix: -yc + x, iy: xc + y, value: 0);
    draw_pixel( ix: -xc + x, iy: yc + y, value: 0);
    std::cout << "(" << x << ", " << y << ")" << std::endl;
}
```



4. bresenham 画圆 算法

输入

(x_0, y_0) R

预处理步骤

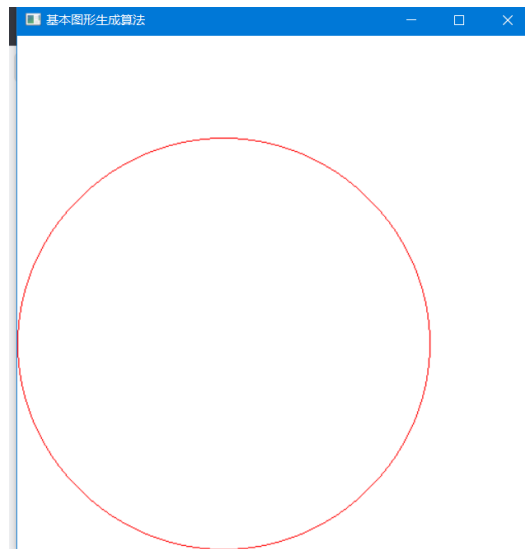
$$x_k := 0 \quad y_k = R$$

$$P_{initial} = 3 - 2R$$

算法

```
while  $x_c < y_c$ 
  if  $P_{initial} < 0$ 
     $(x_k, y_k) \rightarrow (x_k + 1, y_k)$ 
     $P := P + 4 * x_k + 6$ 
  else
     $(x_k, y_k) \rightarrow (x_k + 1, y_k - 1)$ 
     $P := P - 4 * y_k - 4 * x_k + 10$ 
```

```
void bresenham_circle_drawing(int x, int y, int r) {
    int xc = 0, yc = r;
    int p = 3 - 2 * r;
    draw_whole_circle(x, y, xc, yc);
    while (xc < yc) {
        xc++;
        if (p < 0) {
            p = p + 4 * xc + 6;
        } else {
            yc--;
            p = p + 4 * (xc - yc) + 10;
        }
        std::cout << p << " ";
        draw_whole_circle(x, y, xc, yc);
    }
    std::cout << "finish" << " ";
}
```



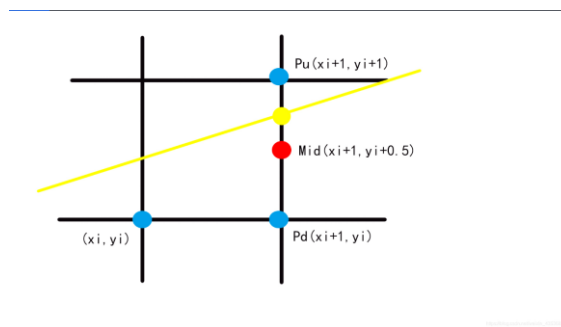
六、实验心得

1. 在本次实验中，使用中点画线法只能绘制斜率在 $0 \leq k \leq 1$ 范围内的直线，为了能够将其推广，我们先介绍中点画线算法的原理。

对于起始点 (x_0, y_0) 、结束点 (x_1, y_1) ，有 $A = y_0 - y_1$ 、 $B = x_1 - x_0$ 、 $C = (x_1 - x_0)(x_0 - y_1)$ 、 $k = (y_1 - y_0)/(x_1 - x_0)$ 。设函数 $F(x, y) = Ax + By + C$ ，则 $F(x, y)$ 将平面分为三部分：

- 1) $F(x, y) = 0$ ，点在直线上
- 2) $F(x, y) > 0$ ，点在直线的上方
- 3) $F(x, y) < 0$ ，点在直线的下方

在画线过程中，对于每个当前像素点 (x_i, y_i) ，每次在最大位移方向，即 x 方向上前进一步，在 y 方向上是否前进取决于前一格中点和直线的位置关系。



如上图所示，可判断中点 Mid 和直线的位置关系，来确定下一点的选取，令 d 为判式：

- 1) $d = F(x_i + 1, y_i + 0.5) = 0$ ， Mid 在直线上，可选取 Pu/Pd ，这里我们统一规定选取 Pd
- 2) $d = F(x_i + 1, y_i + 0.5) > 0$ ， Mid 在直线的上方，直线离 Pd 较近，选取 Pd ，则下一点 y 方向上不变
- 3) $d = F(x_i + 1, y_i + 0.5) < 0$ ， Mid 在直线的下方，直线离 Pu 较近，选取 Pu ，则下一点 y 方向上前进

利用增量思想，每下一步的 d 的判别可以基于本步 d 的正负情况，分本步 $d < 0$ 和 $d \geq 0$ 两种情况讨论。

1) $d < 0$: 本步选取 $P_u(x_i+1, y_i+1)$, 则下一步中点 $(x_i+2, y_i+1.5)$:

$$\begin{aligned} d_{new} &= F(x_i + 2, y_i + 1.5) \\ d_{new} &= A(x_i + 2) + B(y_i + 1.5) + C \\ d_{new} &= [A(x_i + 1) + B(y_i + 0.5) + C] + A + B \\ d_{new} &= d + A + B \end{aligned}$$

2) $d \geq 0$: 本步选取 $P_d(x_i+1, y_i)$, 则下一步中点 $(x_i+2, y_i+0.5)$:

$$\begin{aligned} d_{new} &= F(x_i + 2, y_i + 0.5) \\ d_{new} &= A(x_i + 2) + B(y_i + 0.5) + C \\ d_{new} &= [A(x_i + 1) + B(y_i + 0.5) + C] + A \\ d_{new} &= d + A \end{aligned}$$

在画线过程中每一点的情况都分析完了, 也易得结束画线的条件。接下来只要确定初始值就能顺利地画线了。对于起始点 (x_0, y_0) , 下一步中点为 $(x_0+1, y_0+0.5)$:

$$\begin{aligned} d_0 &= F(x_0 + 1, y_0 + 0.5) \\ d_0 &= A(x_0 + 1) + B(y_0 + 0.5) + C \\ d_0 &= [Ax_0 + By_0 + C] + A + 0.5B \\ d_0 &= A + 0.5B \end{aligned}$$

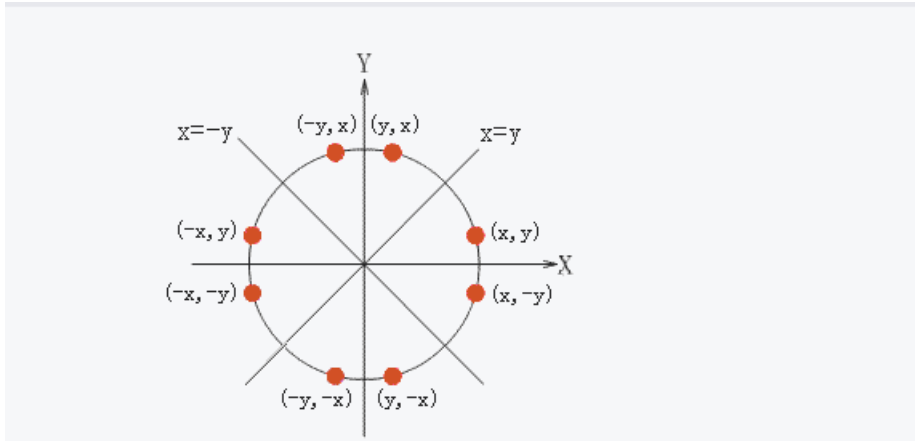
可以发现 d_0 的计算中涉及到了浮点数 0.5, 为了改进我们可以用 $2d_0$ 代替 d_0 , 因为后续计算中只考虑 d 的正负, 则扩大两倍不会对结果有所影响。此时的新 d_0 为:

$$\begin{aligned} d_0 &= 2F(x_0 + 1, y_0 + 0.5) \\ d_0 &= 2A(x_0 + 1) + 2B(y_0 + 0.5) + 2C \\ d_0 &= 2[Ax_0 + By_0 + C] + 2A + B \\ d_0 &= 2A + B \end{aligned}$$

进行推广

$k \in [0, 1]$	$k \in (1, +\infty)$	$k \in [-1, 0)$	$k \in (-\infty, -1)$
$d_0 = 2A + B$	$d_0 = A + 2B$	$d_0 = 2A - B$	$d_0 = A - 2B$
$d < 0$: $d += 2(A+B)$; $x += 1; y += 1$;	$d < 0$: $d += 2B$; $y += 1$;	$d < 0$: $d += 2A$; $x += 1$;	$d < 0$: $d += 2(A-B)$; $y -= 1; x += 1$;
$d \geq 0$: $d += 2A$; $x += 1$;	$d \geq 0$: $d += 2(A+B)$; $y += 1; x += 1$	$d \geq 0$: $d += 2(A-B)$; $x += 1; y -= 1$;	$d \geq 0$: $d -= 2B$; $y -= 1$;

2. 使用中点画圆法或者 bresenham 画圆法, 其实只能画出 $1/8$ 圆弧, 我们是利用圆的八分对称性, 画出整个圆来的。 $(x, -y)$ 、 $(-x, y)$ 、 $(-x, -y)$ 、 (y, x) 、 $(y, -x)$ 、 $(-y, x)$ 、 $(-y, -x)$



七、实验总结

这次实验主要让我们使用两种直线绘制算法（Bresenham 算法和中点画线算法）和两种圆绘制算法（Bresenham 画圆算法和中点画圆算法）绘制图形。

在实验中，我们首先学习了 Bresenham 算法和中点画线算法，然后分别使用这两种算法绘制直线。通过实验我们发现，Bresenham 算法具有更高的效率，能够更快地生成直线，而中点画线算法则更简单易懂。

接下来，我们学习了 Bresenham 画圆算法和中点画圆算法，并分别使用这两种算法绘制圆。实验结果表明，Bresenham 画圆算法和中点画圆算法都能生成圆形，但 Bresenham 画圆算法比中点画圆算法更快。

总的来说，这次实验让我更深入地了解了绘制图形的算法，并通过实际操作加深了对这些算法的理解。这对于我以后在图形学方面的学习和实践将会有很大的帮助。

八、完整代码

```
1 //
2 // Created by 1729 on 2022/11/24.
3 //
4 //A.15 Bresenham 画线算法
5
6 // for drawing a circle
7
8 #include<GL/glut.h>
9 #include <iostream>
10 #include <algorithm>
11
12 #define BLACK 1
13
14 void draw_pixel(int ix, int iy, int value) {
15     glBegin(GL_POINTS);
16     glVertex2i(ix, iy);
17     glEnd();
18 }
19
20 void draw_whole_circle(int x, int y, int xc, int yc) {
21     draw_pixel(xc + x, yc + y, 0);
```